

The Case Against Buying the Circles / ValMia Codebase

A technical valuation review of a proposed ~\$50,000 source-code purchase

SUBJECT

Circles / ValMia — Expo (React Native) social app

ASKING PRICE

~\$50,000 (source code)

PREPARED

29 July 2026

BASIS

Full source audit, 84 source files

STACK

Expo SDK 54 · RN 0.81 · Clerk · Supabase

FINDINGS

4 Critical · ~10 High · ~30 Medium/Low

RECOMMENDATION

Do not buy at \$50,000 as-is.

The codebase is a lightly-customized free starter template whose one differentiator — its trust & safety layer — is also its most broken component, carrying an exploitable "ban any user" flaw. It has zero tests, no production monitoring, and configuration proving it has never run as a real production app. Remediation to a shippable state is weeks of senior engineering work that should be priced *into*, not on top of, any offer.

1 Why this report exists

You asked for the case *against* the purchase. This document makes that case as strongly as the evidence in the code supports — so you can negotiate, re-scope, or walk away with your eyes open. Every claim below is tied to a specific file and line in the delivered source.

The central issue is not any single bug. It is a mismatch between **what you would be paying for** and **what actually exists**. A \$50,000 price implies a differentiated, production-ready product. What the source shows is a modified open-source template whose distinguishing features are the least reliable code in the repository.

2 The seven reasons not to buy

REASON 1 You are buying a starter template, not a product

The `package.json` name is still `clerk-expo-starter`. The Android bundle identifier is still `com.sokratisv.clerkexpostarter` — the *template author's* namespace, never rebranded. The differentiated value is a few thousand lines of feature screens on top of a freely available starter. A competent React Native developer can rebuild the non-trivial parts on the same free base in roughly 4–8 weeks.

`package.json:2 · app.json (android.package)`

REASON 2 A one-request "ban any user" exploit ships in the code

The moderation table's insert policy is `with check (true)`, and a database trigger auto-bans the *reported* user whenever a report's reason is "Harassment or abuse." Nothing verifies the reporter is who they claim to be, and the public API key is committed to the repository. Result, verified in the SQL: anyone can permanently ban any user with a single API call. This is not an isolated bug — it is the security *pattern*: row-level security is switched on but the policies are open (`using (true)`), so blocking, banning, and content-hiding are enforced only inside the app and collapse against a direct API call.

`supabase/ugc_safety_compliance.sql:113–122 · supabase/auto_ban_on_harassment.sql · verified`

REASON 3 The premium features are the weakest code

This app's only real advantage over a generic template is its safety machinery: reporting, blocking, banning, content filters, push. Every one of those has a hole — spoofable blocks, banned users who can un-ban themselves by editing their own profile row, a ban check that "fails open" and admits banned users on any network error, and forgeable push notifications that let anyone spam any user's lock screen. You would be paying a premium for exactly the features that do not work.

`lib/userBlocks.ts · SessionBootstrapContext.tsx:109–117 · functions/send-push`

REASON 4 Zero tests, no CI, no error monitoring

There is not a single test file. The Jest harness is configured but tests nothing, and its script (`jest --watchAll`) cannot even run in CI. There is no linting, no continuous integration, and no crash/error reporting — the app's error boundary logs failures into the void. You would inherit four screens of 1,100–1,700 lines each, riddled with race conditions, and *no safety net* to change them. On a codebase this size, the absence of tests roughly doubles the true cost of every future change.

`no *.test.* files · package.json:10 · src/components/ErrorBoundary.tsx:22`

REASON 5 A live demo cannot reveal the real defects

Several handlers delete or save data, ignore the database's error response, and update the screen anyway. In a demo this looks flawless — the note vanishes, the RSVP toggles. In production the write silently failed and the data reappears on the next refresh. These failures only surface under real authorization rules and load, so any "watch it work" demonstration is *not* evidence the code is sound.

EventDetailScreen.tsx:592 · CircleDetailScreen.tsx:843 · MyProfileScreen.tsx:159

REASON 6 It has never run as a real production app

The production build profile ships a Clerk **test** key (`pk_test_...`) and points authentication at a Clerk *staging* domain. The Google Maps key is still the literal placeholder `YOUR_GOOGLE_MAPS_API_KEY` . This is proof the app has never been properly configured for production — so any claim that it is "live and working" deserves hard scrutiny.

eas.json (build.production) · app.json (associatedDomains, googleMaps.apiKey)

REASON 7 Secrets and identity debt are baked in

Backend configuration and API keys are committed directly into `eas.json` — the very values that `.gitignore` is supposed to keep out — with no rotation path. Session IDs and user emails are written to device logs on every sign-in, unconditionally, in release builds. The project carries three different names across its files. None of this is fatal alone, but together it signals a codebase that was never hardened for handover.

eas.json · logger.ts:1-3 · SignInScreen.tsx:68,101

3 What "fixing it" actually costs

The asking price is not the real price. Below is the minimum remediation required before the app can be responsibly put in front of users. This work is mandatory, not optional.

REMEDIATION ITEM	SEVERITY	ROUGH EFFORT
Re-architect authorization server-side (RLS) — fix ban exploit, blocks, self-unban, notifications	Critical	2–3 weeks
Replace test Clerk key / staging config with real production infrastructure	Critical	2–4 days
Stop logging session IDs & emails; move secrets out of the repo	Critical	1 day
Fix race conditions & silent-failure handlers in the four large screens	High	1–2 weeks
Introduce a test suite, CI pipeline, linting, and crash monitoring	High	1–2 weeks
Version-control the database policies that currently live only in the live DB	High	2–3 days
Dependency cleanup, rebrand, config de-duplication	Medium	2–3 days

Effort assumes one senior React Native / Postgres engineer. Total: on the order of 6–9 weeks of senior work before first responsible launch — a cost that should be credited against, not added to, the purchase price.

The blunt version. You would pay ~\$50,000 for a lightly-customized free template whose one differentiator — trust & safety — is the most broken part, with no tests to protect you while you repair it, and configuration proving it never ran in real production. Add the remediation cost to the sticker and compare it to a clean rebuild on the same starter: the rebuild may be cheaper *and* leave you with code you understand.

4 What would change this recommendation

This is a "not at this price, not without answers" — not an automatic no. The verdict flips toward buying only if:

- **There is a real, verified userbase** you are actually acquiring. Then this becomes a users/acquire deal where code quality matters less — but confirm the users are genuine and engaged.
- **The price drops to reflect remediation.** Treat the 6–9 weeks of security and testing work as a direct credit against the number.
- **You receive a warranty and staged payment** — code escrow or milestone payments contingent on the app passing an independent security review after sale.

5 Questions to put to the seller

Each of these will expose whether there is anything here worth \$50,000:

- "Show me the database row-level security policies for circles, profiles, and notifications." (*They are not in the delivered code.*)
- "Has this ever run on a production `pk_live` Clerk key and real infrastructure? Show me."
- "Where are the automated tests?"
- "How many real, active users are on this today — and can you prove it?"
- "Who owns the Android `com.sokratisv.*` namespace, and can it be transferred?"

Prepared as an independent technical due-diligence review from a full read of the delivered source. Findings are tied to specific files and lines and were current as of the audit date. Two findings (self-unban, forgeable notifications) depend on database policies that exist only in the live backend and could not be confirmed from source — a gap that is itself a reason for caution. This document argues the case against purchase as requested and is not a substitute for a negotiated warranty or a post-sale security review.